FINAL PROJECT REPORT

Classification of Hand Gestures

Using Deep Convolutional Neural Networks

Course	Machine Learning / Deep Learning
Group	Group 2
Framework	PyTorch — fully reproducible pipeline
Dataset	LeapGestRecog	20,000 images	10 gestures	10 subjects
Best Accuracy	~98% (subject-split, fine-tuned CNN)

1. Introduction

Hand gesture recognition — teaching computers to interpret the shapes and movements made with human hands — sits at the intersection of computer vision, human–computer interaction, and deep learning. As natural user interfaces replace physical peripherals, the ability to reliably decode gesture semantics becomes foundational to accessibility, healthcare, gaming, and beyond.

1.1 Motivation and Applications

The following application domains share a common thread: making technology adapt to human behaviour rather than requiring humans to adapt to machines.

Application	Description
Sign Language Translation	Converting hand shapes into text or speech to bridge communication for deaf and hard-of-hearing communities.
Human–Computer Interaction	Replacing keyboards and mice with natural hand movements to make technology more accessible.
VR & Gaming	Creating immersive, controller-free experiences that feel intuitive rather than clunky.
Healthcare	Hands-free control of medical equipment and assistive devices for people with mobility challenges.

1.2 Why Is This Hard?

Although gesture recognition appears trivial to the human visual system, it poses significant engineering challenges:

- Hand variability — different sizes, shapes, finger lengths, skin tones, scars, and calluses.
- Lighting — a closed fist in shadow looks entirely different from one in bright sunlight.
- Cluttered backgrounds — scenes with many objects confuse models trained on clean data.
- Viewpoint and scale — rotation, perspective, and camera distance all alter appearance.
- Real-time speed — a model taking several seconds per frame is operationally useless.
- Subject generalisation — the model must work for people it has never seen during training.

1.3 Research Questions

Core Question

Can deep convolutional neural networks classify hand gestures accurately and efficiently?

Architecture: Are CNNs the right tool, or can simpler models achieve equivalent accuracy?

Components: Which parts of the CNN architecture actually contribute to performance, and which are decorative?

Evaluation methodology: How much does the choice of train/test split strategy change the numbers we report?

2. Dataset

2.1 LeapGestRecog

All experiments use the LeapGestRecog dataset — a publicly available collection of near-infrared hand images captured with a Leap Motion sensor.

Total images	20,000
Gesture classes	10
Subjects	10 different people
Imaging modality	Near-infrared (NIR) — hands appear bright on a dark background
Resolution	240 × 640 pixels (centre-cropped and resized to 128 × 128 for training)

The NIR modality was chosen deliberately: it removes most real-world lighting and background variability present in standard RGB datasets, letting the study focus on architecture and evaluation methodology without getting lost in data-engineering complexity. With 10 subjects each performing all 10 gestures, the dataset provides natural variation in hand shape, size, and style while remaining small enough to train on a laptop GPU.

2.2 The 10 Gesture Classes

Gesture	Description	Difficulty
palm	Open hand, fingers spread	Baseline
L	Thumb up, index pointing sideways	Moderate
fist	Closed hand	Hard — similar to fist_moved
fist_moved	Closed hand at a different angle	Hard — similar to fist

Gesture	Description	Difficulty
thumb	Thumbs up	Moderate — similar to down
index	Pointing with the index finger	Moderate
ok	Thumb and index touching (OK sign)	Moderate
palm_moved	Open hand at a different angle	Hard — similar to palm and c
c	Hand curved like the letter C	Hard — similar to palm_moved
down	Pointing downward	Moderate — similar to thumb

Key Insight

The hardest pairs — (fist, fist_moved), (palm, palm_moved), (thumb, down), (c, palm_moved) — differ only in orientation or slight shape change. These pairs recur throughout the results as the primary confusion points, validating the initial difficulty assessment.

3. Methodology

3.1 The Five Models

A deliberate progression from simplest to most expressive was designed so that each step adds exactly one new capability, making the contribution of each component measurable.

Model	Parameters	Key Addition	Role
Linear Regression	164,000	Single flat layer + MSE loss (wrong loss, on purpose)	Baseline
Logistic Regression	164,000	Cross-entropy loss replaces MSE	+ Correct loss
Multi-Layer Perceptron	~4,200,000	Two hidden layers with ReLU non-linearity	+ Non-linearity
SmallCNN	5,226	Two convolutional blocks — first use of spatial structure	+ Convolutions
Custom CNN	422,986	Four deep conv blocks, BatchNorm, and Dropout	+ Depth & regularisation

3.2 Custom CNN Architecture

The final model follows a standard encoder-classifier design. Spatial resolution halves while the channel count doubles at every block, so the network progressively trades spatial detail for richer feature representations.

Stage	Output Shape	Operations
Input	1 × 128 × 128	Grayscale image
Block 1	32 × 64 × 64	Conv(3×3) → BatchNorm → ReLU → MaxPool(2×2)
Block 2	64 × 32 × 32	Conv(3×3) → BatchNorm → ReLU → MaxPool(2×2)
Block 3	128 × 16 × 16	Conv(3×3) → BatchNorm → ReLU → MaxPool(2×2)
Block 4	256 × 8 × 8	Conv(3×3) → BatchNorm → ReLU → MaxPool(2×2)
AvgPool	256 × 1 × 1	Global average pooling — collapses spatial dims

Stage	Output Shape	Operations
Classifier	10 logits	Linear(256→128) → ReLU → Dropout → Linear(128→10)

Total trainable parameters: 422,986 — big enough to capture real gesture features, small enough to train in minutes on a consumer GPU. This is less than half the size of a typical ImageNet model.

3.3 Training Configuration

All five models were trained under identical settings to ensure a fair comparison. Only the architecture differs.

Framework	PyTorch
Optimizer	Adam with small weight decay
Learning rate	Cosine annealing schedule (initial lr = 1e-3)
Batch size	64 images per step
Epochs	10–20, depending on convergence
Loss function	Cross-entropy (MSE for linear baseline only)
Augmentation	Random horizontal flips, small rotations, slight translations
Tuning	Grid + random search over learning rate, dropout, weight decay

Every run automatically saves its configuration, training curves, and final test results, making all experiments fully reproducible.

3.4 Evaluation Strategies

The project implements and compares two fundamentally different data-splitting strategies, exposing a methodological issue common in published gesture-recognition work.

Strategy	Mechanism	Question Answered	Bias
Random Split (70/15/15)	Images are shuffled globally; every subject's hands appear in all three sets.	Can the model recognise gestures from people it has already seen?	Inflates accuracy — model memorises hands, not gestures
Subject Split	Whole subjects assigned to splits:	Can the model recognise gestures	Realistic estimate of deployment performance

Strategy	Mechanism	Question Answered	Bias
	P0–P6 → train, P7 → val, P8–P9 → test.	from people never seen during training?	

Data Leakage

When the same person's hands appear in both training and test sets (random split), the model can learn person-specific features — hand shape, finger proportions, skin patterns — and exploit them as shortcuts. The gap between the two evaluation methods quantifies this leakage. In this study, that gap is approximately 13 percentage points for the best model.

4. Results

4.1 Model Comparison — Random Split

Model	Test Accuracy	Key Observation
Linear Regression	~60%	Only visually distinct gestures separated; 'ok' acts as a dumping ground
Logistic Regression	~77%	Correct loss alone yields a +17pp gain; visually similar pairs still struggle
Multi-Layer Perceptron	~90%*	Model collapse — predicts 'down' or 'fist_moved' for most inputs; *inflated by leakage
SmallCNN	~97%	Spatial structure matters more than parameter count (800× fewer than MLP)
Custom CNN	~100%	Near-perfect; 100% on a random split is a warning sign, not a celebration

Important Note on MLP

Although the MLP achieves ~90% on the random split, its confusion matrix reveals near-total model collapse: it predicts 'down' or 'fist_moved' for almost everything, correctly identifying only palm (0.72) and down (1.00). The 4M-parameter MLP scores worse than logistic regression on the subject split — 26× fewer parameters in the logistic model.

4.2 Model Comparison — Subject Split

Model	Random Split	Subject Split	Drop (pp)	Interpretation
Linear Regression	~60%	~51%	-9	Already had limited memorisation capacity
Logistic Regression	~77%	~63%	-14	Linear boundaries, moderate leakage
Multi-Layer Perceptron	~90%	~73%	-17	Large but ill-trained; high

Model	Random Split	Subject Split	Drop (pp)	Interpretation
				capacity = high leakage
SmallCNN	~97%	~80%	-17	Spatial features generalise reasonably well
Custom CNN	~100%	~87%	-13	Best generaliser — depth + regularisation helps

Key Takeaway

Deeper models had more to lose — they could memorise more person-specific patterns. But the Custom CNN also generalised best, reaching 87% on unseen people. The SmallCNN's 17pp drop highlights that too few parameters (5K) leads to high training variance and unreliable generalisation, not just low accuracy.

4.3 Per-Class Performance (Custom CNN, Subject Split)

Gesture	Precision	Recall	F1	Note
palm	0.903	0.975	0.938	Strong — distinctive open-palm silhouette
L	0.992	0.935	0.963	Very strong — unique L-shape
fist	0.893	0.790	0.838	Weak — confused with fist_moved
fist_moved	1.000	0.718	0.836	High precision but misses many fist_moved instances
thumb	0.814	0.570	0.671	Weakest — confused with 'down' (both look like a single vertical digit)
index	0.764	0.993	0.863	High recall; some false positives from similar pointers

Gesture	Precision	Recall	F1	Note
ok	0.922	1.000	0.959	Perfect recall — distinctive shape
palm_moved	1.000	0.748	0.856	Perfect precision but low recall
c	0.743	1.000	0.853	Perfect recall; sometimes confused with palm_moved
down	1.000	1.000	1.000	Unique pointing-down silhouette — perfectly distinguished

4.4 Fine-Tuned CNN Results

After hyperparameter search (learning rate, dropout, weight decay, and optimiser), the tuned model achieves 98% accuracy on the subject split — an 11pp improvement over the default configuration.

Optimal learning rate	1e-3
Optimal dropout	0.3
Optimal weight decay	1e-4
Optimal optimiser	Adam
Resulting accuracy	~98% on subject split

Training quality

Tuning did not merely improve accuracy — it made training faster, more stable, and more trustworthy. Both loss curves converge smoothly by epoch 5, with train and validation accuracy tracking closely to near 100%. The previous instability (large val loss spikes, erratic accuracy) disappeared entirely.

5. Analysis

5.1 Ablation Studies

To understand which architectural choices actually drive performance, each component was systematically removed, the model was retrained from scratch, and the accuracy compared against the full model baseline.

Configuration	Accuracy	Effect	Verdict
Full Custom CNN	100%	—	Baseline
No BatchNorm	99.4%	-0.6pp + unstable training curves	Important — smooths gradient flow, stabilises training
No augmentation	99.9%	Essentially no effect on random split	Minor here; expected to matter more on subject split
No Dropout	99.95%	No effect	Decorative — BatchNorm already provides sufficient regularisation

The ablation reveals that BatchNorm is the most important non-convolutional component. Without it, training becomes erratic and final accuracy drops. Dropout, by contrast, is redundant in this configuration: BatchNorm already acts as a regulariser by adding noise through mini-batch statistics during training, so the two mechanisms overlap.

5.2 Grad-CAM Interpretability Analysis

Gradient-weighted Class Activation Mapping (Grad-CAM) was applied to the final convolutional layer to visualise which image regions influenced each classification decision.

- Heatmaps consistently focused on the hand itself, not on background pixels or image artefacts.
- The model learned meaningful gesture-specific features (finger positions, palm curvature) rather than visual shortcuts such as image borders or wrist position.
- Predicted hard pairs (palm vs palm_moved; fist vs fist_moved) showed overlapping activation regions in Grad-CAM, consistent with their geometric similarity.
- Easy gestures (thumb, L, down) produced sharp, localised activations on their distinctive features — the extended thumb, the L-shaped finger arrangement, and the downward-pointing index.
- Even on misclassified examples, the model was attending to the hand. Errors are semantic (misidentifying a shape) rather than localisation failures (looking at the wrong region). This is a healthy failure mode.

Conclusion from Grad-CAM

The CNN is doing the right kind of work — looking at hands, not artifacts. This builds confidence that the model would continue to generalise correctly to new backgrounds and imaging conditions beyond the NIR dataset.

5.3 The Subject Leakage Problem in Detail

Subject leakage is a systematic evaluation flaw that occurs when the same individual's data appears in both the training and test sets. In biometric tasks such as gesture recognition, a person's hand is a consistent visual identity:

- The model can learn person-specific features: proportions, finger lengths, skin patterns, even vein visibility under NIR lighting.
- At test time, seeing the same hand doing a different gesture is much easier than seeing a completely new hand.
- The resulting accuracy number answers the wrong question: it measures recognition of known individuals rather than generalisation to new ones.

Analogy

Subject leakage is analogous to a student who memorises the practice exam questions and then encounters those exact questions on the real exam. The grade looks good but measures familiarity, not understanding.

This issue is not specific to this project — it is pervasive in published gesture-recognition literature. Many state-of-the-art accuracy numbers in the field are inflated by this form of leakage. Subject-independent evaluation (our subject split) is the correct baseline for any deployment-facing claim.

6. Discussion

6.1 Architecture Conclusions

- Convolutional structure matters far more than raw parameter count. The SmallCNN (5K parameters) outperforms the MLP (4M parameters) because convolutions exploit the spatial structure of images — nearby pixels encode edges and shapes, and these local patterns are invariant to translation.
- Each capability step in the progression produced measurable gains: correct loss function (+17pp), non-linearity, convolutions, and depth all contributed.
- BatchNorm is not decorative — it stabilises gradient flow and allows deeper networks to train reliably. Dropout, however, added no benefit on top of BatchNorm for this task and dataset size.

6.2 Evaluation Methodology Conclusions

- Random splits should be avoided for biometric-adjacent tasks. A 13pp inflation in reported accuracy for the Custom CNN (and up to 17pp for other models) is a material difference that changes the practical assessment of whether a system is deployable.
- The gap between random and subject splits is larger for more expressive models — they have higher capacity to memorise person-specific patterns.
- Subject-independent evaluation is the minimum standard for gesture-recognition research. Cross-dataset evaluation (training on one sensor modality, testing on another) would be an even more rigorous test.

6.3 Hyperparameter Tuning Conclusions

Grid and random search over learning rate, dropout, and weight decay yielded meaningful gains. The optimal configuration (lr=1e-3, dropout=0.3, weight decay=1e-4, Adam) improved subject-split accuracy from 87% to ~98% while also making training faster and more stable. This underlines that architecture alone does not determine performance — training configuration matters equally.

7. Conclusions and Future Work

7.1 Summary of Achievements

Achievement	Details
Best accuracy	~98% honest (subject-split, fine-tuned CNN) — exceeds random-split scores of weaker models
Architecture insight	Each capability step — loss, non-linearity, convolutions, depth — contributes measurably

Achievement	Details
Key component	BatchNorm is the most important architectural choice; Dropout is redundant on top of it
Leakage quantified	Random split inflates results by 13pp for the best model; up to 17pp for others
Interpretability	Grad-CAM confirms the model attends to hands, not artifacts; errors are semantic
Reproducibility	All runs auto-save config, training curves, and results; full pipeline in PyTorch

7.2 Limitations

- The NIR modality removes real-world lighting and background challenges — RGB webcam generalisation is unknown.
- 10 subjects is a very small pool; the subject split test set covers only 2 people (P8–P9), limiting statistical power.
- The dataset is balanced and clean; real-world deployments encounter partial occlusions, motion blur, and unbalanced class distributions.
- No temporal modelling — gesture sequences in video are not addressed.

7.3 Future Work

- Transfer learning from pretrained models (ResNet, EfficientNet) likely to push subject-split accuracy past 90% without extra data.
- Temporal modelling with LSTMs or 3D convolutions for continuous gesture sequences in video streams.
- Domain adaptation from near-IR to standard RGB webcam input to support commodity hardware deployment.
- Larger and more diverse datasets with varied lighting, backgrounds, ethnicities, and age groups.
- Cross-dataset evaluation for stronger generalisability claims beyond the LeapGestRecog benchmark.
- Real-time system integration, measuring end-to-end latency and throughput on edge hardware.

References

[1] Hamida, S., et al. LeapGestRecog Dataset. Kaggle, 2018.

[2] LeCun, Y., Bengio, Y., & Hinton, G. Deep learning. Nature, 521(7553), 436–444, 2015.

[3] Ioffe, S., & Szegedy, C. Batch Normalization: Accelerating Deep Network Training. ICML, 2015.

[4] Selvaraju, R. R., et al. Grad-CAM: Visual Explanations from Deep Networks. ICCV, 2017.

[5] Kingma, D. P., & Ba, J. Adam: A Method for Stochastic Optimization. ICLR, 2015.

[6] Srivastava, N., et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014.